

Vulnerability Report of KMS application

1.Cookie Security: Overly Broad Path:

Cookie Security: Overly Broad Path	High
Package: app~^~controller	
src/Controller/AppController.php, line 199 (Cookie Security: Overly Broad Path)	
Issue Details	
Kingdom: Security Features Scan Engine: SCA (Semantic)	
Sink Details	
Sink: setcookie() Enclosing Method: beforefilter() File: src/Controller/AppController.php:199 Taint Flags:	
<pre>196 } else { 197 \$domain = (\$_SERVER['HTTP_HOST'] != 'localhost') ? \$_SERVER['HTTP_HOST'] : 'localhost'; 198 \$appInTime = time(); 199 setcookie('AIT', \$this->Nkn->encrypt_cookie((string) \$appInTime), 0, "/", \$domain, true, true); 200 } 201 202 /* if(!isset(\$_SESSION['userId'])) {</pre>	

Recommendation:

Configure the cookie with the most restrictive **Path** attribute required for its functionality instead of using the root path (/). Restricting the cookie to the specific application directory or module that requires it helps enforce the principle of least privilege, reduces unnecessary exposure of the cookie to unrelated application components, and minimizes the potential impact of cookie theft or misuse. Review the application's requirements and define a narrower path scope wherever feasible.

2.Cross site scripting: DOM

Cross-Site Scripting: DOM	Critical
Package: templates.Dashboard	
templates/Dashboard/index.php, line 189 (Cross-Site Scripting: DOM)	
Issue Details	
Kingdom: Input Validation and Representation Scan Engine: SCA (Data Flow)	
Source Details	
Source: success(0) From: success File: templates/Dashboard/index.php:185	
<pre>182 'ou_name': ou_name 183 }, 184 type: 'post', 185 success: function(res) { 186 if (res == 'No file mapped to this Organisation!') { 187 toastr.error(res); 188 } else {</pre>	
Sink Details	
Sink: ~JS_Generic.html() Enclosing Method: success() File: templates/Dashboard/index.php:189 Taint Flags: WEB, XSS	
<pre>186 if (res == 'No file mapped to this Organisation!') { 187 toastr.error(res); 188 } else { 189 \$('#loadContainer').html(res); 190 } 191 }, 192 error: function(XMLHttpRequest, textStatus, errorThrown) {</pre>	
Package: templates.Form	
templates/Form/edit_index.php, line 286 (Cross-Site Scripting: DOM)	
Issue Details	
Kingdom: Input Validation and Representation Scan Engine: SCA (Data Flow)	
Source Details	
Source: success(0) From: success File: templates/Form/edit_index.php:285	
<pre>282 complete: function(XMLHttpRequest, textStatus) { 283 \$('#loading').fadeOut();</pre>	

Cross-Site Scripting: DOM**Critical**

Package: templates.Form

templates/Form/edit_index.php, line 286 (Cross-Site Scripting: DOM)

```
284 },
285 success: function(dataHtml) {
286 $('#show_size_div').html(dataHtml);
287 }
288 };
```

Sink Details

Sink: ~JS_Generic.html()
Enclosing Method: success()
File: templates/Form/edit_index.php:286
Taint Flags: WEB, XSS

```
283 $('#loading').fadeOut();
284 },
285 success: function(dataHtml) {
286 $('#show_size_div').html(dataHtml);
287 }
288 };
289 }
```

templates/Form/index.php, line 267 (Cross-Site Scripting: DOM)

Issue Details

Kingdom: Input Validation and Representation
Scan Engine: SCA (Data Flow)

Source Details

Source: success(0)
From: success
File: templates/Form/index.php:266

```
263 complete: function(XMLHttpRequest, textStatus) {
264 $('#loading').fadeOut();
265 },
266 success: function(dataHtml) {
267 $('#show_size_div').html(dataHtml);
268 }
269 };
```

Sink Details

Sink: ~JS_Generic.html()

Recommendation

Validate and sanitize all user-controlled data on the server side and properly encode output before inserting it into the DOM. Avoid using dangerous methods such as `.html()`, `.append()`, and `.appendTo()` with untrusted input. Instead, use safer methods such as `.text()` or create DOM elements programmatically. Implement a strict input validation mechanism, output encoding, and Content Security Policy (CSP) to prevent execution of malicious scripts.

3. Cross-Site Scripting: Reflected

Cross-Site Scripting: Reflected	Critical
Package: templates.element.admin	
templates/element/admin/left.php, line 6 (Cross-Site Scripting: Reflected)	
Issue Details	
Kingdom: Input Validation and Representation Scan Engine: SCA (Data Flow)	
Source Details	
Source: Read \$_SERVER['REQUEST_URI'] File: templates/element/admin/left.php:4	
<pre>1 <!-- ===== Sidebar ===== --> 2 3 <?php 4 \$urlpath = substr(\$_SERVER['REQUEST_URI'], strpos(\$_SERVER['REQUEST_URI'], '/') + 1); 5 \$variable = (\$urlpath == 'GlobalRole') ? \$urlpath : ''; 6 echo \$variable; 7</pre>	
Sink Details	
Sink: builtin_echo() File: templates/element/admin/left.php:6 Taint Flags: WEB, XSS	
<pre>3 <?php 4 \$urlpath = substr(\$_SERVER['REQUEST_URI'], strpos(\$_SERVER['REQUEST_URI'], '/') + 1); 5 \$variable = (\$urlpath == 'GlobalRole') ? \$urlpath : ''; 6 echo \$variable; 7 8 \$userrole = \$this->request->getSession()->read('loggedUserInfo')['user_role_mapping'] ['role_id']; 9 ?></pre>	

Recommendation :

Encode all user-controlled data before rendering it in HTML responses using appropriate output encoding functions such as htmlspecialchars() or framework-provided encoding methods. This helps prevent Cross-Site Scripting (XSS) attacks by ensuring that special characters are interpreted as data rather than executable code.

4. HTML5: Overly Permissive CORS Policy:

HTML5: Overly Permissive CORS Policy	Medium
Package: config	
config/bootstrap.php, line 224 (HTML5: Overly Permissive CORS Policy)	
Issue Details	
Kingdom: Encapsulation Scan Engine: SCA (Semantic)	
Sink Details	
Sink: header() File: config/bootstrap.php:224 Taint Flags:	
<pre>221 //Inflector::rules('uninflected', ['dontinflectme']); 222 223 224 header('Access-Control-Allow-Origin: *'); 225 header('Access-Control-Allow-Methods: POST, GET, PUT, PATCH, DELETE, OPTIONS'); 226 header('Access-Control-Allow-Headers: *'); 227 if (\$_SERVER['REQUEST_METHOD'] == 'OPTIONS') {</pre>	

Recommendation

The application is configured with a wildcard CORS policy (Access-Control-Allow-Origin: *), which permits requests from any origin. Restrict the Access-Control-Allow-Origin header to a predefined list of trusted domains that legitimately require access to the application. Avoid using wildcard (*) values, especially for sensitive APIs or authenticated functionality. Additionally, review allowed HTTP methods and headers to ensure that only necessary cross-origin requests are permitted.

5. Insecure Randomness

Insecure Randomness	High
Package: webroot.js	
webroot/js/jquery.treeSelector.js, line 307 (Insecure Randomness)	
Issue Details	
Kingdom: Security Features Scan Engine: SCA (Structural)	
Sink Details	
Sink: FunctionPointerCall: random Enclosing Method: genRandId() File: webroot/js/jquery.treeSelector.js:307 Taint Flags:	
<pre>304 } 305 306 var genRandId = function () { 307 return (new Date()).valueOf() + parseInt(Math.random() * 10000000000, 10) 308 } 309 310 undefined</pre>	

Recommendation

It is recommended to implement the following:

- The application should use "Secure Random()" function instead of using "random()" function.
- Avoid using non-cryptographic random number generators for security purposes.
- Use cryptographically secure random number generators provided by the platform.
- Ensure all tokens, identifiers, or keys used in security contexts are generated securely.
- Review all usages of randomness in the application for potential security impact.
- Replace weak randomness implementations with secure alternatives.

6. Often Misused: File Upload

Often Misused: File Upload	Low
Package: app~^~controller	
src/Controller/AppController.php, line 590 (Often Misused: File Upload)	
Issue Details	
Kingdom: API Abuse Scan Engine: SCA (Semantic)	
Sink Details	
Sink: move_uploaded_file() Enclosing Method: uploaddocument() File: src/Controller/AppController.php:590 Taint Flags:	
<pre>587 // echo \$folder_url;die; 588 if (!file_exists(\$folder_url)) { /* Condition checking File exist or not */ 589 \$full_url = \$folder_url; /* create full filename */ 590 if (move_uploaded_file(\$file->getStream()->getMetadata('uri'), \$full_url)) { 591 return \$filename; 592 } else { 593 return 'permission_error';</pre>	

Often Misused: File Upload	Low
Package: app~^~controller~^~component	
src/Controller/Component/CommonComponent.php, line 158 (Often Misused: File Upload)	
Kingdom: API Abuse Scan Engine: SCA (Semantic)	
Sink Details	
Sink: move_uploaded_file() Enclosing Method: uploaddocument() File: src/Controller/Component/CommonComponent.php:158 Taint Flags:	
<pre>155 // echo \$folder_url;die; 156 if (!file_exists(\$folder_url)) { /* Condition checking File exist or not */ 157 \$full_url = \$folder_url; /* create full filename */ 158 if (move_uploaded_file(\$file->getStream()->getMetadata('uri'), \$full_url)) { 159 return \$filename; 160 } else { 161 return 'permission_error';</pre>	

Recommendation

Implement strict server-side validation for all uploaded files. Validate file extensions, MIME types, file signatures (magic bytes), and file sizes before accepting uploads. Restrict uploads to only approved file types required for business purposes and block executable file formats such as .php, .jsp, .asp, .aspx, .exe, and .sh. Store uploaded files outside the web root whenever possible, rename files using generated identifiers, and enforce appropriate access controls to prevent direct execution of

uploaded content. Additionally, perform malware scanning on uploaded files and maintain detailed logging of upload activities.

7. Password Management: Hardcoded Password

Password Management: Hardcoded Password	High
Package: config	
config/paths.php, line 155 (Password Management: Hardcoded Password)	
Sink Details	
Sink: FieldAccess: srpassword	
File: config/paths.php:155	
Taint Flags:	
<pre>152 define('srhost', "localhost"); 153 define('srport', "5432"); 154 define('srlogin', 'postgres'); 155 define('srpassword', 'postgres'); 156 define('srdatabase', 'WAW_LATEST'); 157 158 undefined</pre>	
config/paths.php, line 147 (Password Management: Hardcoded Password)	
Issue Details	
Kingdom: Security Features	
Scan Engine: SCA (Structural)	
Sink Details	
Sink: FieldAccess: password	
File: config/paths.php:147	
Taint Flags:	
<pre>144 define('host', "localhost"); 145 define('port', "5433"); 146 define('login', 'postgres'); 147 define('password', 'postgres'); 148 define('database_default', 'WAW_LATEST'); 149 150</pre>	
config/app.php, line 422 (Password Management: Hardcoded Password)	
Issue Details	
Kingdom: Security Features	
Scan Engine: SCA (Structural)	
Sink Details	
Sink: ArrayAccess	
File: config/app.php:422	
Taint Flags:	
<pre>419 'timezone' => 'Asia/Kolkata', 420 'host' => 'localhost', 421 'username' => 'postgres', 422 'password' => 'postgres',</pre>	

Issue Title	Severity
Password Management: Hardcoded Password	High
Package: config	
config/app.php, line 422 (Password Management: Hardcoded Password)	
<pre>423 'database' => 'eFile_Common', 424 'flags' => [], 425 'cacheMetadata' => true,</pre>	
config/app.php, line 435 (Password Management: Hardcoded Password)	
Issue Details	
Kingdom: Security Features Scan Engine: SCA (Structural)	
Sink Details	
Sink: ArrayAccess File: config/app.php:435 Taint Flags:	
<pre>432 'host' => 'localhost', 433 //'port' => 9200, 434 'username'=>'elastic', 435 'password'=>'eoffice', 436 'index' => 'catalog_field_mappings', 437 'port' => 443, 438 //'transport' => 'https'</pre>	

Recommendation:

Remove hardcoded password values from source code and configuration files. Store sensitive credentials securely using environment variables, encrypted configuration files, or a dedicated secrets management solution. Implement secure key management practices and ensure passwords are never exposed in application code, client-side files, logs, or repositories. Regularly rotate credentials and enforce least-privilege access controls to reduce the risk of unauthorized access.